



# Amazon RDS — Complete Notes

**Relational Database Service** · Managed SQL databases on AWS

AWS RDS   Service Database   Model Managed   Engines 6

*A reader-friendly, all-in-one reference for Amazon RDS.*

## Table of Contents

1. [What is RDS](#)
2. [Supported Engines](#)
3. [Core Concepts](#)
4. [High Availability \(Multi-AZ\)](#)
5. [Read Replicas \(Scaling Reads\)](#)
6. [Storage Types](#)
7. [Backups, Snapshots & PITR](#)
8. [Security](#)
9. [Amazon Aurora](#)
10. [Maintenance & Scaling](#)
11. [Monitoring](#)
12. [Pricing](#)
13. [Common CLI Commands](#)
14. [RDS vs Other AWS Databases](#)
15. [Quick Mental Model](#)
16. [Common Interview Questions](#)

## 1. What is RDS

Amazon RDS is a **managed relational database** service. AWS handles the heavy operational work — provisioning, OS and DB patching, backups, replication, failover, and scaling — so you focus on schema and queries. You still control the database engine, parameters, and data.

 **Mental model:** RDS is "**managed SQL**." AWS owns the host, OS, and DB engine maintenance; **you** own the data, schema, queries, and tuning. (You do **not** get OS/SSH access.)

**Common uses:** transactional apps (OLTP), web/mobile backends, e-commerce, SaaS, ERP, anything needing a managed SQL database with HA and backups.

## 2. Supported Engines

| Engine        | Notes                                                                          |
|---------------|--------------------------------------------------------------------------------|
| Amazon Aurora | AWS-built, MySQL- & PostgreSQL-compatible; cloud-native performance & scaling. |
| MySQL         | Popular open-source.                                                           |
| PostgreSQL    | Feature-rich open-source.                                                      |
| MariaDB       | MySQL fork.                                                                    |
| Oracle        | Commercial; BYOL or license-included.                                          |
| SQL Server    | Microsoft; multiple editions.                                                  |

Aurora is technically part of the RDS family but is a distinct, re-architected engine — see [section 9](#).

## 3. Core Concepts

| Concept           | Description                                                                      |
|-------------------|----------------------------------------------------------------------------------|
| DB Instance       | The running database environment (compute + memory + storage).                   |
| DB Instance Class | The hardware size (e.g. <code>db.t3.medium</code> , <code>db.r6g.large</code> ). |
| DB Engine         | The database software (MySQL, Postgres...).                                      |
| Endpoint          | DNS name + port apps connect to.                                                 |
| Parameter Group   | Engine configuration settings.                                                   |
| Option Group      | Extra engine features (e.g. Oracle TDE).                                         |
| Subnet Group      | The VPC subnets (across AZs) RDS can use.                                        |
| Multi-AZ          | Synchronous standby for failover (HA).                                           |
| Read Replica      | Async copy for read scaling.                                                     |

## 4. 🚑 High Availability (Multi-AZ)

- **Multi-AZ deployment** maintains a **synchronous standby** replica in a **different AZ**.
- On failure (AZ outage, instance crash, maintenance), RDS **automatically fails over** to the standby — the **endpoint stays the same** (DNS just repoints), typically in 60–120 s.
- The standby is **not** readable (it's for HA, not scaling) — *(except Multi-AZ DB cluster mode, which adds 2 readable standbys)*.
- **Multi-AZ ≠ backup and ≠ read scaling**. It's purely about availability/durability.

## 5. 📖 Read Replicas (Scaling Reads)

- **Asynchronous** copies you can **read from** — offload reporting/read traffic from the primary.
- Up to **15** read replicas (engine-dependent); can be **cross-AZ and cross-region**.
- Each replica has its **own endpoint**; apps must be pointed at it deliberately.
- A replica can be **promoted** to a standalone primary (DR or migration).
- **Replica lag** is possible (async) — don't use for read-after-write consistency.

**Multi-AZ vs Read Replica:** Multi-AZ = synchronous, automatic failover, not readable (HA). Read Replica = asynchronous, manual use, readable (scaling). You often use **both**.

## 6. 💾 Storage Types

| Type                          | Class        | Use                                                               |
|-------------------------------|--------------|-------------------------------------------------------------------|
| General Purpose SSD (gp3/gp2) | SSD          | Most workloads; gp3 lets you scale IOPS/throughput independently. |
| Provisioned IOPS (io1/io2)    | SSD          | I/O-intensive, latency-sensitive production DBs.                  |
| Magnetic                      | HDD (legacy) | Backwards compatibility only — avoid.                             |

- **Storage autoscaling** can grow storage automatically as you fill it.
- Storage is **EBS-backed**; you can scale storage up (not down) without downtime.

## 7. 📀 Backups, Snapshots & PITR

- **Automated backups** — daily full + transaction logs; retention **0–35 days**. Enable **Point-in-Time Recovery (PITR)** to restore to any second in the window.
- **Manual snapshots** — user-triggered; kept until you delete them (survive instance deletion).
- Backups/snapshots are stored in **S3** (managed by AWS) and can be **copied cross-region** and **shared** across accounts.
- Restoring a snapshot **creates a new instance** (you then repoint the endpoint).

⚠️ Automated backups are deleted when the instance is deleted (unless you take a final snapshot). **Manual snapshots persist** — use them before risky changes.

## 8. 🗝️ Security

- **Network:** launch in a **VPC**, control access with **Security Groups**; keep DBs in **private subnets** (no public access).
- **Auth:** native DB users/passwords, or **IAM database authentication** (token-based), or **Kerberos/AD**.
- **Encryption at rest:** **KMS** (enable at creation — can't easily add later; encrypts storage, backups, snapshots, replicas).
- **Encryption in transit:** SSL/TLS to the endpoint.
- **Secrets:** store/rotate credentials in **Secrets Manager** (native RDS rotation).
- **Audit:** CloudTrail (API) + engine audit logs to CloudWatch.

## 9. 🚀 Amazon Aurora

AWS's cloud-native engine, MySQL- and PostgreSQL-compatible:

- **Storage auto-grows** up to 128 TiB; data is **6-way replicated across 3 AZs** automatically.
- Up to **15 low-latency read replicas**; failover in seconds via a **cluster (writer/reader) endpoint**.
- **Aurora Serverless v2** — autoscales capacity (ACUs) up/down with load; great for variable/spiky workloads.
- **Global Database** — sub-second cross-region replication for global apps & DR.
- Typically **3–5x MySQL / 2–3x PostgreSQL** throughput; pay for storage actually used.

Choose **Aurora** for performance, automatic storage scaling, and fast failover; choose **standard RDS engines** for exact version parity, lower entry cost, or specific feature needs.

## 10. 🛠️ Maintenance & Scaling

- **Maintenance window** — weekly window for patching; some changes need a brief restart.
- **Vertical scaling** — change the **instance class** (brief downtime, or use Multi-AZ to minimize).
- **Storage scaling** — increase storage / IOPS online; **autoscaling** optional.
- **Read scaling** — add **read replicas**; **Aurora Serverless** scales compute automatically.
- **Blue/Green Deployments** — create a synchronized staging copy, test, then switch over with minimal downtime.

## 11. 📊 Monitoring

- **CloudWatch metrics** — CPU, connections, freeable memory, IOPS, storage, replica lag.
- **Enhanced Monitoring** — OS-level metrics at up to 1-second granularity.
- **Performance Insights** — visual DB load + top SQL/wait analysis for tuning.
- **Event subscriptions** (SNS) — failovers, low storage, backups, etc.

- **CloudTrail** — management API audit.

---

## 12. 💰 Pricing

---

You pay for:

1. **DB instance hours** — by class & engine; On-Demand or **Reserved Instances** (1–3 yr, big savings).
2. **Storage** — per GB-month provisioned (+ provisioned IOPS for io1/io2).
3. **Backup storage** — free up to DB size; extra beyond that.
4. **Data transfer OUT** and **cross-region** replication.
5. **Multi-AZ** roughly doubles instance/storage cost (standby).
6. **Extras** — Performance Insights (long retention), snapshots export, etc.

🔑 **Cost tips:** use **Reserved Instances** for steady prod · right-size the class · **Graviton ( g )** instances for price/perf · stop non-prod (up to 7 days at a time) · trim old snapshots · use **Aurora Serverless** for spiky/dev workloads.

---

## 13. 📄 Common CLI Commands

---

```
aws rds create-db-instance --db-instance-identifier mydb \
  --db-instance-class db.t3.micro --engine postgres \
  --master-username admin --master-user-password '****' \
  --allocated-storage 20 --vpc-security-group-ids sg-123 --multi-az

aws rds describe-db-instances
aws rds create-db-snapshot --db-instance-identifier mydb --db-snapshot-identifier snap1
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier mydb-restored --db-snapshot-identifier snap1
aws rds create-db-instance-read-replica \
  --db-instance-identifier mydb-replica --source-db-instance-identifier mydb
aws rds modify-db-instance --db-instance-identifier mydb \
  --db-instance-class db.r6g.large --apply-immediately
aws rds reboot-db-instance --db-instance-identifier mydb --force-failover
aws rds delete-db-instance --db-instance-identifier mydb --final-db-snapshot-identifier final-snap
```

## 14. RDS vs Other AWS Databases

| Service                                       | Type                               | When                                                                 |
|-----------------------------------------------|------------------------------------|----------------------------------------------------------------------|
| RDS                                           | Managed relational (SQL)           | Traditional OLTP apps needing a specific engine.                     |
| Aurora                                        | Cloud-native relational            | High performance/scale relational with auto-storage & fast failover. |
| DynamoDB                                      | Managed NoSQL (key-value/document) | Massive scale, single-digit-ms, serverless, flexible schema.         |
| ElastiCache                                   | In-memory (Redis/Memcached)        | Caching, sessions, leaderboards, sub-ms reads.                       |
| Redshift                                      | Data warehouse (OLAP)              | Analytics over huge datasets.                                        |
| DocumentDB / Neptune / Keyspaces / Timestream | Purpose-built                      | Documents / graphs / Cassandra / time-series.                        |

## 15. Quick Mental Model

- **RDS = managed SQL.** AWS runs the host/OS/engine maintenance; you own data + queries (no SSH).
- Pick an **engine** (MySQL, Postgres, MariaDB, Oracle, SQL Server, or **Aurora**).
- **Multi-AZ** = synchronous standby for **automatic failover** (HA), same endpoint — *not* readable, *not* a backup.
- **Read Replicas** = async, readable copies for **scaling reads** (own endpoint, possible lag).
- **Backups** (automated + PITR) and **manual snapshots** (persist) protect data — snapshot before risky changes.
- Keep it **private in a VPC**, encrypt with **KMS**, manage creds in **Secrets Manager**.
- **Aurora** for cloud-native scale/perf; **Reserved Instances + right-sizing** for cost.

## 16. Common Interview Questions

Rapid-fire questions interviewers ask about RDS:

- **Q: Multi-AZ vs Read Replica?** — Multi-AZ = synchronous standby for **automatic failover** (HA), same endpoint, not readable. Read Replica = asynchronous, **readable** copy for scaling reads (own endpoint, possible lag).
- **Q: Does Multi-AZ improve read performance?** — No. It's for availability/durability only; use read replicas to scale reads.
- **Q: Automated backup vs snapshot?** — Automated = daily + transaction logs, enables PITR, deleted with the instance. Manual snapshot = user-triggered, **persists** until deleted.
- **Q: What is Amazon Aurora?** — AWS cloud-native MySQL/Postgres-compatible engine; storage auto-grows, 6-way replicated across 3 AZs, fast failover, up to 15 replicas.
- **Q: How do you secure RDS?** — Private subnets, security groups, KMS encryption at rest (set at creation), TLS in transit, Secrets Manager for credentials, IAM auth optional.
- **Q: Can you SSH into an RDS instance?** — No — it's managed; you get a DB endpoint, not OS access.
- **Q: How to scale an RDS database?** — Vertically (change instance class), storage autoscaling, read replicas for reads, or Aurora Serverless for variable load.
- **Q: How to migrate with minimal downtime?** — Blue/Green Deployments or AWS DMS.

---

 *Notes compiled as a quick reference for Amazon RDS.*